Università della Svizzera italiana

Faculty of Informatics

Machine Learning

Homework Report #1

Baio Federico

8 October 2025

1 Ridge Regression Exercise

1.1 Point a

We start by briefly defining the linear regression; where the relationship between the input and the target variable is expressed as a polynomial function

$$y(x, \mathbf{w}) = \sum_{j=0}^{M} w_j x^j.$$

We aim to then use y as an estimator for t (our set of observation is defined as $(x_n, t_n)_{n=1,...,N}$). Then, after fitting the set of observations, the goal is to find the value of our coefficients $\mathbf{w} = \{w_j\}_{j=0,...,M}$ that minimize the loss function. Now, the formulation of the ridge regression model is used as method to reduce the overfitting, and in the case of linear regression, this is done by optimizing an error (loss) function, adjusted with a regularization term (λ):

$$\tilde{E}(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^{N} \frac{1}{2}(y(x_n, \mathbf{w}) - t_n)^2 + \frac{\lambda}{2}||\mathbf{w}||_2^2$$

1.2 Point b

Assumed a set of training data: $\mathcal{O} = \{(\mathbf{x}_n, t_n)\}_{n=1,...,N}$ we define the Mean squared error (MSE) or loss function as:

$$E(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^{N} \frac{1}{2}(y(x_n, \mathbf{w}) - t_n)^2$$

However, since we are interested in deriving the optimal regression parameters for a ridge regression problem, the MSE that we are going to minimize is the one defined above in the point a:

$$\tilde{E}(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^{N} \frac{1}{2}(y(x_n, \mathbf{w}) - t_n)^2 + \frac{\lambda}{2}||\mathbf{w}||_2^2$$

So first of all, since we want to find the coefficients $\mathbf{w} = \{w_j\}_{j=0,...,M}$ that minimize this function, we need to take the gradient w.r.t to $\mathbf{w}$,

$$\nabla_{\mathbf{w}} \tilde{E} = \frac{\partial \tilde{E}}{\partial w_j} = \frac{\partial}{\partial w_j} \frac{1}{N} \sum_{n=1}^{N} \frac{1}{2}(y(x_n, \mathbf{w}) - t_n)^2 + \frac{\partial}{\partial w_j} \frac{\lambda}{2}||\mathbf{w}||_2^2$$

which, reminding that

$$y(x, \mathbf{w}) = \sum_{j=0}^{M} w_j x^j = \mathbf{w}^T \boldsymbol{\phi}(x)$$

and $\boldsymbol{\phi}(x)$ is the feature vector, equal to $(\phi^{(0)}(x), ..., \phi^{(M)}(x))^T = (x^0, x^1, \ldots, x^M)^T$, the partial derivative above is equal to

$$\frac{1}{N} \sum_{n=1}^{N} (\mathbf{w}^T \phi(x_n) - t_n)\phi(x_n) + \lambda \mathbf{w} = \frac{1}{N} \left(\mathbf{w}^T \sum_{n=1}^{N} \phi(x_n)\phi(x_n)^T - \sum_{n=1}^{N} t_n \phi(x_n)^T \right) + \lambda \mathbf{w}$$

Then, using the Design Matrix $\mathbf{\Phi}$ and the target vector $\mathcal{T} = (t_1, ..., t_N)^T$ will lead us to:

$$\frac{1}{N} \left(\mathbf{w}^T \mathbf{\Phi}^T \mathbf{\Phi} - \mathcal{T}^T \mathbf{\Phi} \right) + \lambda \mathbf{w}$$

Since we want to minimize this value, to find the closed form solution, we need to set the gradient to zero, transposing everything and factor out $\mathbf{w}$

$$0 \stackrel{!}{=} \frac{1}{N}\left(\mathbf{w}^T\mathbf{\Phi}^T\mathbf{\Phi} - \mathcal{T}^T\mathbf{\Phi}\right) + \lambda\mathbf{w} \iff (\mathbf{\Phi}^T\mathbf{\Phi} + N\lambda I)\mathbf{w} = \mathbf{\Phi}^T\mathcal{T}$$

Since this is a linear system of $(M+1)$ equations for $(M+1)$ variables we will assume that there is a unique solution given by

$$\mathbf{w}_{\text{min}} = (\mathbf{\Phi}^T\mathbf{\Phi} + N\lambda I)^{-1}\mathbf{\Phi}^T\mathcal{T}$$

where since the MSE was defined with $\frac{1}{N}$, the regularization term is multiplied by N. We can get the same information (not standardized by N) evaluating:

$$\mathbf{w}_{\text{min}} = (\mathbf{\Phi}^T\mathbf{\Phi} + \lambda I)^{-1}\mathbf{\Phi}^T\mathcal{T}$$

1.3 Point c

As stated above, we use ridge regression as a possible solution to the overfitting problem. In particular, the introduction of the regularization term aims at constraining the model parameters. The value of λ therefore influences (and constrains) $\mathbf{w}_{\text{min}}$:

- **If $\lambda \to 0$:** In this case the penalization applied is basically negligible, leaving an overfitting problem and not reducing the coefficients. Especially if $\lambda = 0$, the constraint on the value of $\mathbf{w}_{\text{min}}$ is essentially removed, so that the ridge regression simply reduces to the ordinary least squares estimate:

$$\mathbf{w}_{\text{min}} = (\mathbf{\Phi}^T\mathbf{\Phi})^{-1}\mathbf{\Phi}^T\mathcal{T}.$$

- **If $\lambda \to \infty$:** The constraint (and shrinkage) on the model parameters increases, so that the denominator in the closed form solution becomes very large, to the point that the ridge regression coefficients approach 0. With respect to the first case, here we have the opposite problem. Indeed, the coefficients lead us to a model with serious underfitting problem; essentially an almost constant model.

1.4 Point d

To resolve the problem and prove that the value of θ indicated in the problem statement are not optimal we can use the formula evaluated above to get the actual values ($\mathbf{w}_{\text{min}}$ or θ_{min})that minimize the MSE with $\lambda = 0.8$:

$$\theta_{\text{min}} = (\mathbf{\Phi}^T\mathbf{\Phi} + N\lambda I)^{-1}\mathbf{\Phi}^T\mathcal{T}$$

To do that we define the matrix $\mathbf{\Phi}$ and the vector $\mathcal{T}$:

$$\mathbf{\Phi} = \begin{bmatrix} 1 & 1 & 2 \\ 1 & 2 & 1 \\ 1 & 3 & 3 \\ 1 & 4 & 2 \end{bmatrix} \quad \mathcal{T} = \begin{bmatrix} 10 \\ 12 \\ 18 \\ 21 \end{bmatrix}$$

where in the matrix $\mathbf{\Phi}$ we add as first column a column of 1 to account for the value of θ_0.

We start by computing the following operations (see the code in appendix A):

$$\mathbf{\Phi}^T\mathbf{\Phi} = \begin{bmatrix} 4 & 10 & 8 \\ 10 & 30 & 21 \\ 8 & 21 & 18 \end{bmatrix}, \quad N\lambda I = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 3.2 & 0 \\ 0 & 0 & 3.2 \end{bmatrix}$$

(note that we have zero in the position $1,1$ of $N\lambda I$ because we do not penalize the intercepts θ_0)[1]) and then we sum them and invert the result

$$(\mathbf{\Phi}^T\mathbf{\Phi} + N\lambda I)^{-1} = \begin{bmatrix} 1.57804995 & -0.26416907 & -0.33381364 \\ -0.26416907 & 0.12487992 & -0.02401537 \\ -0.33381364 & -0.02401537 & 0.19692603 \end{bmatrix}$$

so that, we can get the actual optimal regression estimator for this model by multiplying all for $\mathbf{\Phi}^T\mathcal{T}$:

$$\theta_{\min} = (\mathbf{\Phi}^T\mathbf{\Phi} + N\lambda I)^{-1} \begin{bmatrix} 61 \\ 172 \\ 128 \end{bmatrix} = \begin{bmatrix} 8.09582133 \\ 2.29106628 \\ 0.71325648 \end{bmatrix}$$

So, the actual parameters that minimize the MSE are actually:

$$\theta_0 \approx 8.1 \quad \theta_1 \approx 2.29 \quad \theta_2 \approx 0.71$$

To confirm that we can evaluate the MSE for the claimed parameters and the MSE for the actual optimal parameters (look the code in appendix B.
For the claimed ones we have $\theta = [1, 2, 1]$ so that for example for the first observation:

$$(y(x_1, \theta) - t_1)^2 = ((1 + 1 * 2 + 2 * 1) - 10)^2 = 25$$

we can to that for all the observation, then sum the result, divide by N and add the adjust with the regularization term, so that we will ends up with:

$$\tilde{E}_{\text{claimed}}(\theta) = \frac{1}{N}\sum_{n=1}^{N}\frac{1}{2}(y(x_n, \theta) - t_n)^2 + \frac{\lambda}{2}||\theta||_2^2 = 30.525$$

we can do the same thing also for the optimal parameters calculated above:

$$\tilde{E}_{\text{optimal}}(\theta) = \frac{1}{N}\sum_{n=1}^{N}\frac{1}{2}(y(x_n, \theta_{\min}) - t_n)^2 + \frac{\lambda}{2}||\theta_{\min}||_2^2 = 3.7243335734870313 \approx 3.72$$

which is significantly less then the MSE for the claimed parameters, confirming that the parameters that the problem statement suggest are surely not the best ones.

Note that in both the evaluations, in the second (penalization) part we penalize as said above only the coefficients, not the intercepts

[1] in the ridge regression if we look at the penalization part is written also as $\lambda \sum_{j=1}^{p} \beta_j^2$, with j that start from 1 because the intercept β_0 adjusts the model so that its predictions are centered around the mean of y, so must not be penalized

2 Kernel Methods Exercise

2.1 Point a

The main point in the kernel methods, that allows extending linear algorithms to solve even non linear problems, is obviously the use of the Kernel function:

$$k(\mathbf{x}_n, \mathbf{x}_m) = \phi(\mathbf{x}_n)^T \phi(\mathbf{x}_m) = \langle \phi(\mathbf{x}_n), \phi(\mathbf{x}_m) \rangle$$

As we can see above, instead of explicitly computing the feature map $\phi(x)$, which can be computationally expensive or infinite-dimensional for non-linear problems, we can substitute this value with a Kernel function that captures the same inner product and respect the following rules:

- symmetric, since we aim to substitute $\phi(\mathbf{x}_n)^T \phi(\mathbf{x}_m)$, so $k(\mathbf{x}_n, \mathbf{x}_m) = k(\mathbf{x}_m, \mathbf{x}_n)$
- The gram matrix $\mathbf{K} = (k(\mathbf{x}_n, \mathbf{x}_m))_{n,m=1,\dots,N}$ must be positive semi-definite, i.e., for all vectors $\mathbf{c} \neq 0$, $\mathbf{c}^T \mathbf{K} \mathbf{c} \geq 0$, in that case the kernel is called valid kernel

The most important thing that allows us to use the kernel is the fact that it can be shown that

$$\mathbf{w} = \Phi^T \mathbf{a}$$

and so the coefficients are a linear combination of the feature of the training data. And so substituting $\mathbf{w} = \Phi^T \mathbf{a}$ in the formula of the loss function, one can easily arrive to express the model as:

$$y(x) = \mathbf{k}(\mathbf{x})^T \mathbf{a}$$

where $\mathbf{a}$ represent the weights of the training points $\mathbf{x}$ in the prediction $y(x)$ and is obtained minimizing the loss function w.r.t $\mathbf{a}$:

$$\mathbf{a} = (\mathbf{K} + \lambda I_N)^{-1} \mathcal{T}$$

The main point is exactly that as we can see, in this formulation of the model $y(x)$ we never directly use the feature vector $\phi(\mathbf{x})$. We instead use the kernel function, that if valid, allows us to compute the scalar product in the high-dimensional feature space without ever explicitly computing the features. In this way we can apply any linear algorithm in a feature space where the data may become linearly separable, even if the relationship is non-linear in the original space. In fact, the feature can even have an infinite number of dimension, as in the famous RBF (radial basis function) kernel:

$$K(\mathbf{x}, \mathbf{y}) = \exp\left(-\frac{||\mathbf{x} - \mathbf{y}||^2}{2\sigma^2}\right)$$

where the exponential can be expanded leading us to an obviously incomputable feature space form with infinite number of dimension:

$$\phi(x) = \left[\exp\left(-\tfrac{1}{2}\|x\|^2\right) \ \frac{x_1^{n_1} \cdot x_k^{n_k}}{\sqrt{n_1! \cdot n_k!}}\right]_{(n_1,\dots,n_k)\in\mathbb{N}^k}.$$

2.2 Point b

To show that $k(\mathbf{x}, \mathbf{y}) = (\mathbf{x}^T\mathbf{y})^d$ with $d \geq 1$ is a valid kernel, we must check the rules stated above

- **Symmetric**= being the inner product commutative, is straightforward to see that $k(\mathbf{x}, \mathbf{y}) = (\mathbf{x}^T\mathbf{y})^d = (\mathbf{y}^T\mathbf{x})^d = k(\mathbf{x}, \mathbf{y})$
- **Positive semi-definite**: to show that a kernel is valid we must choose a feature $\phi(x)$ such that we can write $k(\mathbf{x}, \mathbf{y}) = \langle \phi(x), \phi(y) \rangle$. For example in this case if $d = 1$ is immediate that $x^T y = \langle x, y \rangle$ as definition of inner product (which is by definition positive semi-definite). But we can show that also for $d > 1$.
  For example a simple case can be with $\mathbf{x}, \mathbf{y} \in \mathbb{R}^2$ and $d = 2$ so that

  $$(\mathbf{x}^T\mathbf{y})^2 = (x_1y_1 + x_2y_2)^2 = x_1^2y_1^2 + 2x_1x_2y_1y_2 + x_2^2y_2^2$$

  and we can find for example the features

  $$\phi(x) = (x_1^2, \sqrt{2}(x_1x_2), x_2^2)$$

  for which we can write $(x^T y)^2$ as

  $$\langle \phi(x), \phi(y) \rangle$$

  We can also continue, showing that for any $d \geq 1$ a suitable feature map $\phi(x)$ can be constructed. By definition so, for all vectors $\mathbf{c} \neq 0$, $\mathbf{c}^T\mathbf{K}\mathbf{c} \geq 0$:

  $$\left\langle \sum_i c_i\phi(\mathbf{x}), \sum_j c_j\phi(\mathbf{y}) \right\rangle \geq 0$$

Therefore $k(\mathbf{x}, \mathbf{y}) = (\mathbf{x}^T\mathbf{y})^d$ is a valid kernel for any $d \geq 1$

2.3 Point c

The problem with the kernel method is however that even if we simplify the calculations and allows us to use valid kernels when the feature space $\phi(x)$ is complicated, we must store all the training points. In fact while the inner product shows the similarity between feature vector, the kernel shows the similarity between training points in the feature space. So is easy to see that comparing the memory requirement, the kernel ridge regression scales worse with the number of data points. In fact, to confirm this, we see that since the model became

$$y(\mathbf{x}) = \mathbf{k}(\mathbf{x})^T(\mathbf{K} + \lambda I_N)^{-1}\mathcal{T}$$

now we do not evaluate the inverse of a MxM matrix (where M is the number of features) but a NxN matrix where N is the number of data points! Clearly, inverting a matrix when $N >>$ becomes computationally infeasible.

3 Support Vector Machine

3.1 Point a

First of all we plot all the points

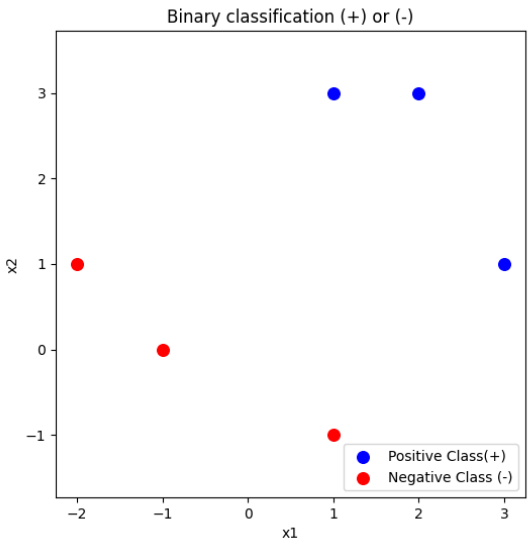

Figure 1

It is easy to answer the second part of the question because, since we are able to draw a line that separate the two groups, we can confidently say that the classes are linearly separable.

3.2 Point b

To find the equation of the maximized margin hyperplane (in this case a straight line) we remind that the margin is twice the distance from the decision boundary to the nearest point of either class, so that we have at least one point of each class which push against the margin. To find the equation for the decision boundary we note that (since the exercise requires a solution by inspection) the two nearest points are $n_1 = (1, -1)$ and $n_2 = (3, 1)$ and the decision boundary is probably a line with negative slope. The key point is that to find the straight line that could be our decision boundary and maximize the margin, this line should:

- passes through the middle point between them
- should be orthogonal to the line that connect our two points

so that $n_1 = (1, -1)$ and $n_2 = (3, 1)$ becomes support vector.
We start by finding the slope of the line that passes through them using the formula:

$$m_{\text{ort}} = \frac{y_2 - y_1}{x_2 - x_1} = \frac{1 - (-1)}{3 - 1} = 1$$

so that reminding the relation between the slope of two orthogonal line we can find:

$$m_{\text{dec_bound}} = -\frac{1}{m_{\text{ort}}} = -1$$

After that we find the middle point:

$$p = (\frac{1+3}{2}, \frac{-1+1}{2}) = (2, 0)$$

so that we can derive the straight line that passes trough that point and have slope $m_{\text{dec_bound}} = -1$ using:

$$y - y_p = m(x - x_p) \iff y = -x + 2$$

So that, using our notation we ends up with:

$$x_2 = -x_1 + 2$$

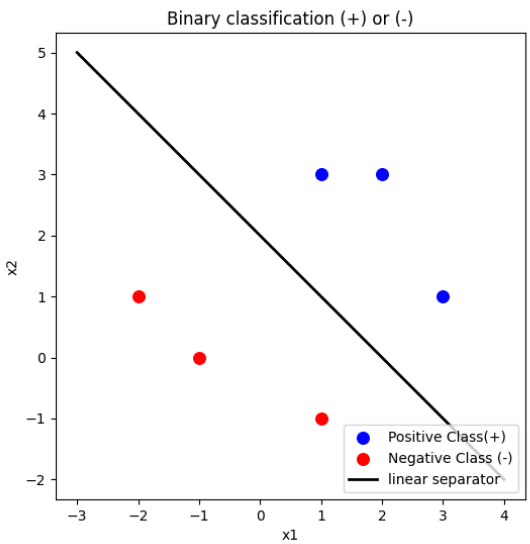

Figure 2

Note that maybe there could be a better straight line that divide the two groups, using Lagrangian multiplier to maximize the margin, but by inspection this is the one that look better.
So, to evaluate the weights $\mathbf{w}$ and the bias term b we write the discriminant function in our case:

$$y(\phi) = \mathbf{w}^\top \phi + b$$

where since we are in the 2D space $\phi = x_1$ and $y(\phi) = x_2$ so that we ends up with:

$$x_1 + x_2 - 2 = 0$$

with

$$\mathbf{w} = \begin{pmatrix} 1 \\ 1 \end{pmatrix} \quad \text{and} \quad b = -2$$

We can conclude by saying that, the support vectors point are $n_1 = (1, -1)$ for the $(-)$ class and $n_2 = (3, 1)$ for the $(+)$ class. They are the support vector since are the ones that push against the maximum margin hyperplane in our solution (remember though

that the maximum margin hyperplane has been derived by inspection).
We also add that, to verify our solution we can show that for a normalized $y(\phi)$, all data points,

$$|y(\phi)| = |\mathbf{w}^\top \phi + b| \geq 1$$

i.e.

$$\bullet \text{ if } \mathbf{w}^\top \phi + b \geq 1, \qquad \bullet \text{ if } \mathbf{w}^\top \phi + b \leq -1$$

and our support vector (if we normalize the value of $\mathbf{w}$ and b) satisfy the conditions:

- $y(\phi) = \mathbf{w}^\top \phi + b = 1 \longrightarrow$ for the support vector of class $(+)$
- $y(\phi) = \mathbf{w}^\top \phi + b = -1 \longrightarrow$ for the support vector of class $(-)$

By standardizing, to confirm the argument above, we ends up with the normalized value for $\mathbf{w}$ and for b:

$$\mathbf{w} = \begin{pmatrix} 0.5 \\ 0.5 \end{pmatrix} \quad \text{and} \quad b = -1$$

but we also see that there is another point that respect the condition to be defined as a support vector stated above,(i.e. $y(\phi) = \mathbf{w}^\top \phi + b = 1$). The point is $n_3 = (1, 3)$ and so it can be considered a support vector for the $(+)$ class.
Since the other points have value $|y(\phi)| > 1$ for the $(+)$ class or $|y(\phi)| > 1$ for the $(-)$ class, we can say that the only support vectors are the ones mentioned above.

3.3 Point c

The margin width for our solution (normalized) is so given, by definition, by:

$$M = \frac{2}{\|\mathbf{w}\|} = \frac{2}{\sqrt{\mathbf{w}^\top \mathbf{w}}} = \frac{2}{\sqrt{0.5^2 + 0.5^2}} \approx 2.828$$

3.4 Point d

If we apply a polynomial kernel of degree 2 to the dataset the points will surely remain linearly separable. In fact, the kernel trick is useful because allow non linearly separable points to be represented in a linear separable way, representing the original point in a feature space ϕ. So if the points in this dimension are linearly separable, they will also be linearly separable even in higher dimension using the polynomial kerne of degree 2. Regarding the margin width, it might probably change because, even if the formula does not directly depends from the feature chosen ϕ, the value of the vector $\mathbf{w}$ depend on the choice of the feature, and so is defined in higher dimensions compared to before. Indeed, setting derivatives of the Lagrangian $\mathcal{L}$ to zero with respect to $\mathbf{w}$ yields the condition

$$\mathbf{w} = \sum_{n=1}^{N} a_n t_n \phi_n,$$

So the margin width may change, since we can more easily find better linear separator in higher dimensions (if instead even in the higher dimension the optimal solution remain the one selected before, will not change)

3.5 Point e

Adding another point $n_7 = (2, 0.5)$ will lead us to an updated plot

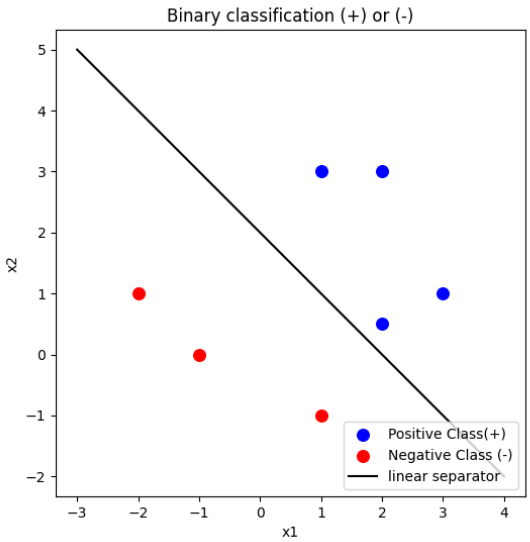

Figure 3

In which the result evaluated before would be no longer optimal, as we can see in the figure 3, with our line evaluated before. In fact, even if the classes remain linearly separable, the new point land inside the margin. Therefore, if we want to evaluate again the maximum margin hyperplane the points indicated before ($n_2 = (3, 1)$ and $n_3 = 1, 3)$) would not be anymore the support vectors for the class $(+)$. In fact we will end up finding a new straight line probably less inclined (smaller absolute value of the slope) since the new two points used to construct it will be $n_1 = (1, -1)$ for the $(-)$ class as before but the new point $n_7 = (2, 0.5)$ for the $(+)$ class. All that will leads us to a definitely smaller width of the margin, since the two points are closer.

If we add slack variables ξ to construct a soft-margin SVM so that the new constraint became

$$t_n y(\phi_n) = t_n(\mathbf{w}^\top \phi_n + w_0) \geq 1 - \xi_n, \quad \xi_n \geq 0.$$

we might be able to keep the previous solution, that remains a valid classifier, but no longer the optimal one. In fact, this method is used for non linearly separable dataset, whether we think that some of the data have been misclassified or correctly classified, but lie inside the margin. In that last case, for a normalized SVM,

$$0 < \xi_n \leq 1$$

and the point ($n_7 = (2, 0.5)$ in our case) become a support vector.

A Code to calculate the minimized coefficients

```
import numpy as np
A = np.array([
    [1,1, 2],
    [1,2, 1],
    [1,3, 3],
    [1,4, 2]
])
T=np.array([
    [10],
    [12],
    [18],
    [21]
])
Lambda=0.8
N=4

AtA=A.T@A
NLamI=np.diag([0, 1, 1])*Lambda*N
print(AtA) #product matrix Phi*Phi^T
print(NLamI) #product matrix N*Lambda*I

inverse=np.linalg.inv(AtA+NLamI)
print(inverse) # inverse matrix (Phi*Phi^T+N*Lambda*I)

w_min=inverse@(A.T@T)
print(w_min) # result vector min coefficient w
```

B Code to evaluate the MSE

```
theta = np.array([1,2,1])
#theta = np.array ([8.09582133, 2.29106628, 0.71325648])
# select the values of theta depending on which MSE you want to evaluate,
# first the claimed, second the ones calculated above

y_hat = A @ theta # prediction

MSE_ridge = ((1/N)*(np.sum(0.5*(T.T - y_hat)**2)) + (Lambda/2) * np.sum(theta[1:]**2))
print(MSE_ridge) #MSE
```